

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



XÂY DỰNG WEBSITE TÍCH HỢP CHATBOT
HỖ TRỢ TƯ VẤN TUYỂN SINH ĐẠI HỌC IUH

Ngành: Hệ Thông Tin

Giảng viên hướng dẫn: Nguyễn Huỳnh nam

Danh sách thành viên nhóm 5

1. Nguyễn Trọng Phú (NT)	19445851
2. Vũ Văn Phồ	19440861
3. Đặng Duy Khang	19438581
4. Nguyễn Lê Trung Nghĩa	19441671
5. Nguyễn Văn Phúc	19443131

THÀNH PHỐ HỒ CHÍ MINH, 22 THÁNG 10 NĂM 2022

MỤC LỤC

CHƯƠNG 1	MỞ ĐẦU	4
I.	Giới thiệu đề tài	4
1.1.	Lý do thực hiện đề tài	4
1.2.	Phạm vi nghiên cứu	4
1.2.1.	Phạm vi không gian	4
1.2.2.	Phạm vi thời gian	4
1.2.3.	Phạm vi nội dung	4
1.3.	Xác định bài toán	5
1.4.	Phương pháp tiến hành	5
II.	Mô tả bài toán	5
CHƯƠNG 2	TỔNG QUAN LÝ THUYẾT ĐỀ TÀI	7
I.	CHATBOT	7
1.1.	Khái niệm	7
1.2.	Retrieval based Chatbots	7
1.3.	Generative based Chatbots	7
II.	THƯ VIỆN TENSORFLOW	7
1.1.	Khái niệm	7
1.2.	<i>Áp dụng trong bài toán</i>	8
III.	THƯ VIỆN KERAS	8
1.1.	Khái niệm	8
1.2.	<i>Áp dụng trong bài toán</i>	8
IV.	THƯ VIỆN PICKLE	9
1.1.	Khái niệm	9
1.2.	<i>Áp dụng trong bài toán</i>	9

V.	THƯ VIỆN NLTK'	9
1.1.	Khái niệm	9
1.2.	Áp dụng trong bài toán	9
VI.	THƯ VIỆN FLASK	10
1.1.	Khái niệm	10
1.2.	Áp dụng trong bài toán	10
VII.	KHÁI NIỆM LIÊN QUAN	10
1.1.	Overfitting	10
1.2.	Relu activation	11
1.3.	Softmax Activation	11
	CHƯƠNG 3 QUY TRÌNH SỐ HÓA VÀ XỬ LÝ DỮ LIỆU.	13
I.	ĐIỀU KIỆN TIỀN QUYẾT CHO	13
II.	MỘT SỐ THỨ CẦN CHUẨN BỊ CHO BÀI TOÁN	13
1.1.	Thu thập dữ liệu	13
1.2.	Xây dựng cấu trúc cây thư mục	14
III.	QUY TRÌNH CHUẨN HÓA VÀ XỬ LÝ DỮ LIỆU	15
1.1.	Năm bước chính để thực hiện	15
1.2.	Quy trình chuẩn hóa và xử lý	16
1.2.1.	Import và load tệp dữ liệu	16
1.2.2.	Tiền xử lý dữ liệu	17
1.2.3.	Tạo tập dữ liệu đào tạo về kiểm tra	17
1.2.4.	xây dựng mô hình	18
1.2.5.	Dự đoán phản hồi (trên giao diện người dùng)	19
	CHƯƠNG 4 XÂY DỰNG MÔ HÌNH VÀ ĐÁNH GIÁ	21
I.	XÂY DỰNG MÔ HÌNH	21
1.1.	xây dựng mô hình	21

1.2. Tối ưu cho mô hình	21
II. ĐÁNH GIÁ MÔ HÌNH.....	22
1.1. Loss Function	22
CHƯƠNG 5 THỰC NGHIỆM VÀ ĐÁNH GIÁ.....	24
I. THỰC NGHIỆP HỆ THỐNG.....	24
II. ĐÁNH GIÁ QUÁ TRÌNH HOẠT ĐỘNG CHO HỆ THỐNG	24
III. KIỂM THỬ VÀ CÁC VẤN ĐỀ GẶP PHẢI TRONG QUÁ TRÌNH THỰC HIỆN	24

CHƯƠNG 1

MỞ ĐẦU

I. Giới thiệu đề tài

1.1. Lý do thực hiện đề tài

Hiện nay, công tác tuyển sinh đại học của các trường ngày càng được chú trọng hơn, đặc là các phương pháp để có thể giải đáp thắc mắc, tiếp cận đến sinh viên một cách tốt nhất. Phương pháp hiệu quả để làm việc này đó chính là xây dựng một ứng dụng Chatbots AI để hỗ trợ công tác tuyển sinh vừa thuận tiện cho phía nhà trường vừa có thể giải đáp thắc mắc cho các sinh viên một cách kịp thời. Theo như tìm hiểu thì trường Đại học Công Nghiệp TP HCM vẫn đang thực hiện tư vấn tuyển sinh bằng các phương pháp cũ như gọi điện thoại, nhắn tin,... từ các trang truyền thông của trường. Sử dụng các phương pháp này tốn rất nhiều thời gian, nguồn lực của nhà trường nhưng cũng không mang lại hiệu quả cao vì sinh viên không nhận được phản hồi một cách kịp thời, việc kiểm soát thông tin được truyền tải đến sinh viên gặp nhiều khó khăn. Vì vậy thấy được nhu cầu cần thiết của phía nhà trường nhóm đã tiến hành nghiên cứu và xây dựng ứng dụng chatbots AI hỗ trợ công tác tuyển sinh trường Đại học Công Nghiệp TP HCM.

1.2. Phạm vi nghiên cứu

1.2.1. Phạm vi không gian

Xây dựng website tích hợp chatbots AI hỗ trợ tuyển sinh tại Trường Đại học Công Nghiệp TP. HCM

1.2.2. Phạm vi thời gian

Bài nghiên cứu này được thực hiện từ tháng 08 năm 2022 cho đến 22 tháng 10 năm 2022.

1.2.3. Phạm vi nội dung

Có được một bài báo cách xây dựng một chatbots

Tìm hiểu về cách hoạt động của một số thư viện như: Tensorflow, Keras, Pickle, Nltk, Flask

Xây dựng mô hình mạng Neural cho đề tài

1.3. *Xác định bài toán*

Một ứng dụng chatbots điều quan trọng nhất là có thể trả lời đúng câu hỏi mà người dùng nhập vào và đây cũng chính là bài toán đặt ra cho đề tài. Hiện nay có nhiều cách để xây dựng một chatbots AI nhưng phổ biến nhất là sử dụng machine learning. Machine Learning là một nhánh của trí tuệ nhân tạo (AI), nó là một lĩnh vực nghiên cứu cho phép máy tính có khả năng cải thiện chính bản thân chúng dựa trên dữ liệu mẫu (training data) hoặc dựa vào kinh nghiệm (những gì đã được học). Machine learning có thể tự dự đoán hoặc đưa ra quyết định mà không cần được lập trình cụ thể. Phương pháp học máy được sử dụng trong chatbots AI là học có giám sát (Supervised machine learning) được đào tạo với các tập dữ liệu được gán nhãn, cho phép các mô hình học và phát triển chính xác hơn theo thời gian. Để xây dựng được tập dữ liệu huấn luyện phải xác định được dữ liệu đầu vào và dữ liệu đầu ra, tiến hành làm sạch dữ liệu và huấn luyện. Tiếp theo là sẽ xây dựng mô hình và load mô hình được đào tạo, sau đó sử dụng giao diện người dùng để dự đoán phản hồi từ bot. Mô hình sẽ cho biết lớp mà câu hỏi thuộc về từ đó xác định lớp và sau đó lấy ra câu phản hồi ngẫu nhiên từ trong lớp đó.

1.4. *Phương pháp thiện*

Trong dự án này nhóm sẽ xây dựng chatbot bằng cách sử dụng các kỹ thuật Deep learning là Supervised. Chatbot sẽ được training trên tập dữ liệu chứa các mẫu đầu vào và các phản hồi tương ứng. Sử dụng mạng LSTM để phân loại thông điệp của người dùng thuộc danh mục nào và sau đó sẽ trả về phản hồi ngẫu nhiên từ danh sách các phản hồi. Bây giờ, hãy tạo một chatbot sử dụng NLTK, Keras, python... NLTK (bộ công cụ ngôn ngữ tự nhiên) là một thư viện Python phổ biến được sử dụng cho NLP (Natural Language Processing).

II. **Mô tả bài toán**

Trong những năm gần đây, mặc dù còn mới mẻ trong lĩnh vực khoa học và công nghệ nhưng Chatbot đang được nghiên cứu và phát triển với tốc độ nhanh chóng bởi các trung tâm nghiên cứu, trường đại học và đặc biệt là các sàn thương mại điện tử... với rất nhiều ứng dụng trên các lĩnh vực khác nhau: giải trí, y tế và, thương mại, tự động hóa,... kể từ khi chatbots được phát triển mạnh mẽ vào năm 2016, nó đã nhanh chóng trở thành một trong những xu hướng công nghệ được quan tâm nhất trong giai đoạn đó đến nay. Trên thực tế cho thấy, sự gia tăng của chatbot đã dẫn đến dự đoán của Gartner (công ty nghiên cứu và tư vấn về vấn đề công nghệ thông tin)

Chatbot tuyến sinh sẽ phát triển dựa trên sự kết hợp của các kịch bản có trước và tự học trong quá trình tương tác. Ta sẽ tương tác với chatbot qua nền tảng tin nhắn. với các câu hỏi được đặt ra, chatbots sử dụng các hệ thống xử lý ngôn ngữ tự nhiên (Natural Language Processing) để phân tích dữ liệu sau đó chúng lựa chọn các thuật toán học máy để đưa ra các loại phản hồi khác nhau. Chúng sẽ dự đoán và phản hồi chính xác nhất có thể. Chatbots sử dụng nhiều hệ thống quét từ khóa đầu vào, sau đó bot khởi động một hành động, kéo một câu trả lời với các từ khóa phù hợp nhất và trả lời thông tin từ một cơ sở dữ liệu / API, hoặc bàn giao cho con người. Nếu tình huống chưa xảy ra (không có dữ liệu), chatbot sẽ trả lời bằng một mệnh đề liên quan gần nhất.

CHƯƠNG 2 TỔNG QUAN LÝ THUYẾT ĐỀ TÀI.

I. CHATBOT

1.1. Khái niệm

Chatbot là một phần mềm thông minh có khả năng giao tiếp và thực hiện các hành động tương tự như con người. Chatbots được sử dụng rất nhiều trong tương tác với khách hàng, tiếp thị trên các trang mạng xã hội và trả lời tin nhắn tự động với khách hàng...

1.2. Retrieval based Chatbots

Retrieval-based chatbot sử dụng các mẫu đầu vào và các phản hồi tương ứng đã được định nghĩa từ trước. Sau đó, nó sử dụng một số loại phương pháp tiếp cận heuristic để chọn câu trả lời thích hợp nhất với đầu vào (input) và bối cảnh (context). Heuristic ở đây có thể đơn giản là các kỹ thuật khớp luật (rule-based expression matching). Retrieval-based chatbot không thể tạo ra bất kỳ từ mới nào khác, chúng chỉ lấy một số câu trả lời từ một tập có sẵn.

1.3. Gennerative based Chatbots

Những mô hình như thế này không dựa vào tập phản hồi định nghĩa trước. Chúng dựa trên mạng nơ-ron Sequence to Sequence, được phát triển dựa trên bài toán Dịch Máy (Machine translation). Nhưng thay vì dịch từ ngôn ngữ này sang ngôn ngữ khác, mô hình này sẽ dịch từ input ra output, tức là chúng có thể truy xuất thông tin từ input và phản hồi lại một cách ấn tượng như bạn đang nói chuyện với con người. Generative based Chatbot sẽ cần một lượng lớn dữ liệu và training trên mạng Deep Neural.

II. THƯ VIỆN TENSORFLOW

1.1. Khái niệm

TensorFlow là một thư viện mã nguồn mở phục vụ cho hoạt động machine learning, được phát triển bởi các nhà nghiên cứu từ Google. Hỗ trợ mạnh mẽ các phép toán học để tính toán trong machine learning và deep learning điều này đã giúp việc tiếp cận các bài toán trở nên đơn giản, nhanh chóng và tiện lợi hơn

Trong quy trình phát triển một phần mềm bất kỳ đòi hỏi rất nhiều đoạn mã cũng như thuật toán được triển khai. Thuật toán vừa để phân tích, tổng hợp dữ liệu vừa là

nền tảng để phần mềm có thể khởi chạy. Tuy nhiên chương trình càng lớn thì khối lượng phép toán càng nhiều. Cách tính toán thủ công không thể đảm bảo hiệu suất như mong muốn được. Tensorflow như một chương trình hỗ trợ tính toán bằng cách tiếp cận mạnh mẽ các phép tính và bài toán phức tạp. Nhờ có Tensorflow, người dùng có thể đơn giản hóa toán học thông qua các đồ thị luồng dữ liệu tổng hợp.

1.2. Áp dụng trong bài toán

Dùng trong việc import SGD một hàm giúp tối ưu cho model

III. THƯ VIỆN KERAS

1.1. Khái niệm

Keras là một mã nguồn mở cho Neural Network được viết bởi ngôn ngữ Python. Nó là một thư viện được phát triển vào năm 2005 bởi Francois Chollet, là một kỹ sư nghiên cứu Deep Learning. Keras có thể sử dụng chung với các thư viện nổi tiếng như Tensorflow, CNTK, Theano. Một số ưu điểm của Keras như: Dễ sử dụng, dùng đơn giản hơn Tensor, xây dựng model nhanh. Chạy được trên cả CPU và GPU. Hỗ trợ xây dựng CNN, RNN hoặc cả hai.

1.2. Áp dụng trong bài toán

Trong đề tài thư viện keras được áp dụng để xây dựng model bằng Sequential model:

```
# Create model - 3 layers. First layer 128 neurons, second layer 64 neurons and 3rd output layer contains number of neurons
# bằng số ý định để dự đoán ý định đầu ra với softmax
model = Sequential()
model.add(Dense(128, input_shape=(len(train_x[0]),), activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.5))
model.add(Dense(len(train_y[0]), activation='softmax'))
# Biên dịch mô hình. Độ dốc ngẫu nhiên với gradient gia tốc Nesterov mang lại kết quả tốt cho mô hình này
sgd = SGD(lr=0.01, decay=1e-6, momentum=0.9, nesterov=True)
model.compile(loss='categorical_crossentropy', optimizer=sgd, metrics=['accuracy'])
#lập và lưu mô hình
hist = model.fit(np.array(train_x), np.array(train_y), epochs=200, batch_size=5, verbose=1)
model.save('chatbot_model.h5', hist)
print("model created")
```

- Đầu tiên sẽ khởi tạo model Sequential cho bài toán
- Dense là cách khai báo 1 layer trong Keras, với số chiều đầu ra của layer đó và activation là hàm kích hoạt của layer.
- Đầu ra của bài toán sẽ sử dụng hàm activation để tính xác suất là số gì.
- Hàm compile là hàm dùng để xác định các optimizer trong quá trình train.

- Hàm `fit()`: Bao gồm data train, test đưa vào training. `Batch_size` thể hiện số lượng mẫu mà Mini-batch GD sử dụng cho mỗi lần cập nhật trọng số. `Epoch` là số lần duyệt qua hết số lượng mẫu trong tập huấn luyện.

IV. THƯ VIỆN PICKLE

1.1. Khái niệm

Pickle được sử dụng để thực hiện chuyển đổi các cấu trúc đối tượng Python sang một dạng byte để có thể được lưu trữ trên ổ đĩa hoặc được gửi qua mạng. Sau đó, luồng ký tự này sau đó có thể được truy xuất và chuyển đổi trở lại sang dạng đối tượng ban đầu trong Python.

1.2. Áp dụng trong bài toán

Sau khi chuyển đổi có thể ghi hoặc load vào hầu hết đối tượng của các kiểu dữ liệu phổ biến như dữ liệu kiểu `.pkl`

```
words = pickle.load(open('words.pkl', 'rb'))
classes = pickle.load(open('classes.pkl', 'rb'))
```

V. THƯ VIỆN NLTK

1.1. Khái niệm

Nói một cách đơn giản, NLTK là thư viện dùng để xử lý ngôn ngữ tự nhiên (NLP) là phát triển các ứng dụng và dịch vụ có khả năng hiểu ngôn ngữ của con người. Ở đây chúng ta đang nói về các ví dụ thực tế về xử lý ngôn ngữ tự nhiên (NLP) như nhận dạng giọng nói, dịch lời nói, hiểu các câu hoàn chỉnh, hiểu các từ đồng nghĩa của các từ phù hợp và viết các câu và đoạn văn hoàn chỉnh đúng ngữ pháp.

1.2. Áp dụng trong bài toán

NLTK trong ứng dụng chatbots AI được dùng để tiền xử lý dữ liệu như:

Modul Tokenization có thể được định nghĩa là quá trình chia nhỏ văn bản đã cho, tức là chuỗi ký tự thành các đơn vị nhỏ hơn được gọi là mã thông báo. Các mã có thể là các từ, số hoặc dấu chấm câu. Nó còn được gọi là phân đoạn từ. Mã hóa từng từ:

```
w = nltk.word_tokenize(pattern)
words.extend(w)
```

Loại bỏ các từ trùng lặp, sắp xếp lại các lớp và gắn tag chong từng dữ liệu.

```
# lemmatize, lower từng từ và loại bỏ các từ trùng lặp
words = [lemmatizer.lemmatize(w.lower()) for w in words if w not in ignore_words]
words = sorted(list(set(words)))
# Sắp xếp lại các lớp
classes = sorted(list(set(classes)))
# documents = sự kết hợp giữa các mẫu và ý định
print (len(documents), "documents")
# classes = ý định
print (len(classes), "classes", classes)
# words = tất cả các từ, từ vựng
print (len(words), "unique lemmatized words", words)
```

Và một số cách làm sạch dữ liệu như:

```
def clean_up_sentence(sentence):
    # tokenize the pattern - split words into array
    sentence_words = nltk.word_tokenize(sentence)
    # stem each word - create short form for word
    sentence_words = [lemmatizer.lemmatize(word.lower()) for word in sentence_words]
    return sentence_words
```

VI. THƯ VIỆN FLASK

1.1. *Khái niệm*

Flask là một web frameworks, nó thuộc loại micro-framework được xây dựng bằng ngôn ngữ lập trình Python. Flask cho phép xây dựng các ứng dụng web từ đơn giản tới phức tạp. Nó có thể xây dựng các api nhỏ, ứng dụng web chẳng hạn như các trang web, blog, trang wiki hoặc một website dựa theo thời gian hay thậm chí là một trang web thương mại. Flask cung cấp cho các công cụ, các thư viện và các công nghệ hỗ trợ làm những công việc trên.

1.2. *Áp dụng trong bài toán*

Dùng để xử lý các request từ client đến server và trả về respon trên giao diện.

VII. KHÁI NIỆM LIÊN QUAN

1.1. *Overfitting*

Overfitting là hiện tượng mô hình tìm được quá khớp với dữ liệu training. Việc quá khớp này có thể dẫn đến việc dự đoán nhầm lẫn, và chất lượng mô hình không còn tốt trên dữ liệu test nữa. Dữ liệu test được giả sử là không được biết trước, và không được sử dụng để xây dựng các mô hình Machine Learning.

1.2. Relu activation

Hàm ReLU đang được sử dụng khá nhiều trong những năm gần đây khi huấn luyện các mạng neuron. ReLU đơn giản lọc các giá trị < 0 . Nhìn vào công thức chúng ta dễ dàng hiểu được cách hoạt động của nó. Một số ưu điểm khá vượt trội của nó so với Sigmoid và Tanh:

- (+) Tốc độ hội tụ nhanh hơn hẳn. ReLU có tốc độ hội tụ nhanh gấp 6 lần Tanh. Điều này có thể do ReLU không bị bão hoà ở 2 đầu như Sigmoid và Tanh.
- (+) Tính toán nhanh hơn. Tanh và Sigmoid sử dụng hàm exp và công thức phức tạp hơn ReLU rất nhiều do vậy sẽ tốn nhiều chi phí hơn để tính toán.
- (-) Tuy nhiên ReLU cũng có một nhược điểm: Với các node có giá trị nhỏ hơn 0, qua ReLU activation sẽ thành 0, hiện tượng này gọi là “Dying ReLU”. Nếu các node bị chuyển thành 0 thì sẽ không có ý nghĩa với bước linear activation ở lớp tiếp theo và các hệ số tương ứng từ node này cũng không được cập nhật với gradient descent. => Leaky ReLU ra đời.
- (-) Khi learning rate lớn, các trọng số (weights) có thể thay đổi theo cách làm tất cả neuron dừng việc cập nhật.

1.3. Softmax Activation

Softmax Function dịch ra Tiếng Việt là hàm trung bình mũ. Hàm softmax tính toán xác suất xảy ra của một sự kiện. Nói một cách khái quát, hàm softmax sẽ tính khả năng xuất hiện của một class trong tổng số tất cả các class có thể xuất hiện. Sau đó, xác suất này sẽ được sử dụng để xác định class mục tiêu cho các input.

Cụ thể, hàm softmax biến vector k chiều có các giá trị thực bất kỳ thành vector k chiều có giá trị thực có tổng bằng 1. Giá trị nhập có thể dương, âm, bằng 0 hoặc lớn hơn 1, nhưng hàm softmax sẽ luôn biến chúng thành một giá trị nằm trong khoảng $(0;1]$.

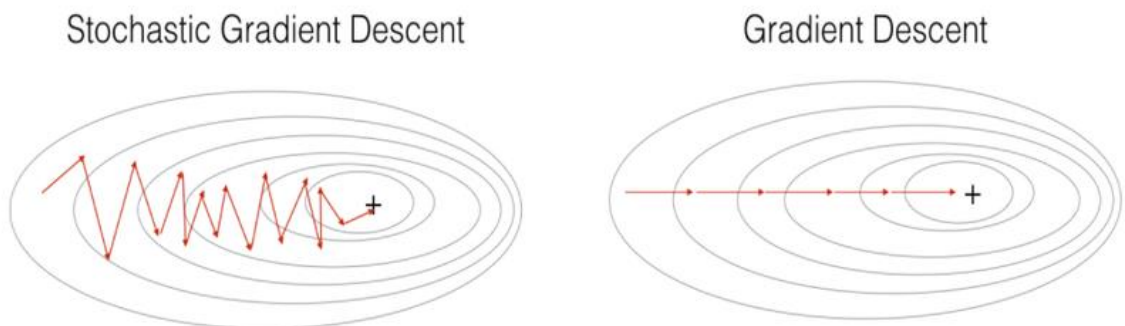
Như vậy, chúng có thể được gọi là “xác suất”. Nếu một trong các giá trị nhập rất nhỏ hoặc âm, hàm softmax biến chúng thành 1 xác suất nhỏ. Còn nếu một giá trị nhập lớn thì nó sẽ được chuyển thành một xác suất lớn. Nhưng xác suất luôn lớn hơn 0 và nhỏ hơn 1, hoặc bằng 1.

- Hàm Softmax được sử dụng trong mô hình hồi quy logistic phân loại.

- Trong xây dựng neural network (mạng thần kinh nhân tạo), hàm softmax được sử dụng trong nhiều lớp.

1.4. Stochastic Gradient Descent (SGD)

Stochastic là 1 biến thể của Gradient Descent. Thay vì sau mỗi epoch chúng ta sẽ cập nhật trọng số (Weight) 1 lần thì trong mỗi epoch có N điểm dữ liệu chúng ta sẽ cập nhật trọng số N lần. Nhìn vào 1 mặt, SGD sẽ làm giảm đi tốc độ của 1 epoch. Tuy nhiên nhìn theo 1 hướng khác, SGD sẽ hội tụ rất nhanh chỉ sau vài epoch. Công thức SGD cũng tương tự như GD nhưng thực hiện trên từng điểm dữ liệu.



Nhìn vào 2 hình trên, ta thấy SGD có đường đi khá là zig zắc, không mượt như GD. Dễ hiểu điều đó vì 1 điểm dữ liệu không thể đại diện cho toàn bộ dữ liệu. Đặt câu hỏi tại sao phải dùng SGD thay cho GD mặc dù đường đi của nó khá zig zắc? Ở đây, GD có hạn chế đối với cơ sở dữ liệu lớn (vài triệu dữ liệu) thì việc tính toán đạo hàm trên toàn bộ dữ liệu qua mỗi vòng lặp trở nên cồng kềnh. Bên cạnh đó GD không phù hợp với online learning. Vậy online learning là gì? online learning là khi dữ liệu cập nhật liên tục (ví dụ như thêm người dùng đăng kí) thì mỗi lần thêm dữ liệu ta phải tính lại đạo hàm trên toàn bộ dữ liệu => thời gian tính toán lâu, thuật toán không online nữa. Vì thế SGD ra đời để giải quyết vấn đề đó, vì mỗi lần thêm dữ liệu mới vào chỉ cần cập nhật trên 1 điểm dữ liệu đó thôi, phù hợp với online learning.

CHƯƠNG 3 QUY TRÌNH SỐ HÓA VÀ XỬ LÝ DỮ LIỆU.

I. ĐIỀU KIỆN TIỀN QUYẾT CHO

- Cài đặt python phiên bản 3.10.8
- Cài đặt 1 số thư viện cần thiết: tensorflow, keras, pickle, nltk, flask
- Chạy trên phần mềm Visual Studio Code
- Hệ điều hành Window

II. MỘT SỐ THỨ CẦN CHUẨN BỊ CHO BÀI TOÁN

1.1. Thu thập dữ liệu

Giới thiệu bộ dữ liệu: Bộ dữ liệu được thu thập và biên soạn tập dữ liệu từ web, phòng truyền thông IUH(ICC)... của trường đại học công nghiệp bao gồm 286 câu hỏi và 293 câu trả lời liên quan đến các vấn đề như tư vấn học vụ, tham vấn học đường, điểm xét tuyển, chương trình đào tạo, giáo dục...

Để thử nghiệm độ tin cậy của mô hình dự đoán, nhóm thu thập tập dữ liệu gồm 447 câu hỏi, trong đó có 235 câu hỏi (có dấu tiếng Việt). dựa trên nhiều nhóm ngành khác nhau dựa trên các tiêu chí như nhóm ngành, điểm số, khối xét tuyển để thực nghiệm.

Dữ liệu được nhận dạng và lưu trữ dưới dạng file .JSON bao gồm 3 yếu tố:

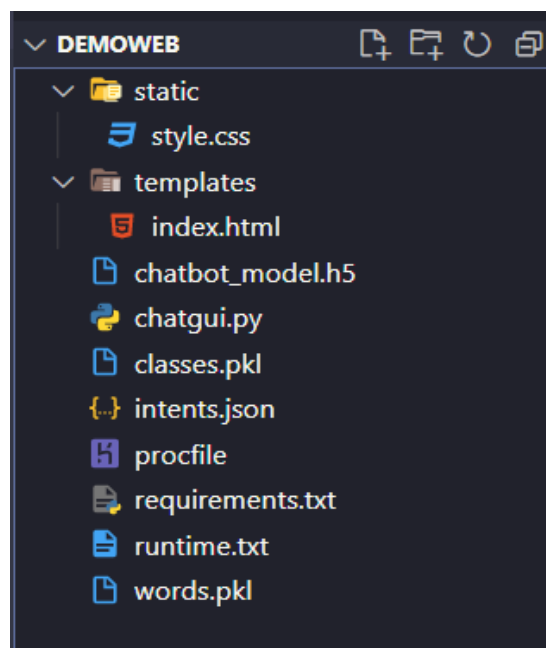
- **Tag:** là nhãn lớp cho nội dung nhập của người dùng
- **Patterns:** mẫu câu đầu vào được training để phân lớp
- **Responses:** các câu trả lời (bot) được mapping để hồi đáp những request trước đó.

```
intents.json X
intents.json > ...
17 {"tag": "thủ_tục_xét_tuyển_phương_thức_1",
18 "patterns": ["thủ tục xét tuyển phương thức 1", "thủ tục xét tuyển thẳng"],
19 "responses": ["Hồ sơ gồm có:\n-Phiếu đăng ký xét tuyển (theo mẫu trên website của Trường);\n-01 Bản photocopy Học bạ THPT"],
20 "context": [""]}
21 },
22 {"tag": "thủ_tục_xét_tuyển_phương_thức_2",
23 "patterns": ["thủ tục xét tuyển phương thức 2", "thủ tục xét tuyển kết quả học tập THPT lớp 12"],
24 "responses": ["Hồ sơ:\n-Phiếu đăng ký xét tuyển;\n-01 Bản photocopy Học bạ THPT (có công chứng);\n-01 Bản photocopy Bản"],
25 "context": [""]}
26 },
27 {"tag": "thủ_tục_xét_tuyển_phương_thức_3",
28 "patterns": ["thủ tục xét tuyển phương thức 3", "thủ tục xét tuyển sử dụng kết quả kỳ thi tốt nghiệp"],
29 "responses": ["Hồ sơ, lệ phí đăng ký xét tuyển theo quy định của Bộ Giáo dục và Đào tạo, với mã tuyển sinh của Trường THPT"],
30 "context": [""]}
31 },
32 {"tag": "thủ_tục_xét_tuyển_phương_thức_4",
33 "patterns": ["thủ tục xét tuyển phương thức 4", "thủ tục xét tuyển Sử dụng kết quả kỳ thi đánh giá năng lực của Đại học"],
34 "responses": ["Hồ sơ gồm có:\n-Phiếu đăng ký xét tuyển theo mẫu quy định của Nhà trường;\n-01 Bản photocopy Học bạ THPT"],
35 "context": [""]}
36 },
37 {"tag": "địa_điểm_nộp_hồ_sơ",
38 "patterns": ["địa điểm nộp hồ sơ", "địa điểm nộp"],
39 "responses": ["Địa điểm nộp hồ sơ: Nộp trực tiếp tại Trường hoặc Nộp qua đường bưu điện. "],
40 "context": [""]}
41 },
```

hình minh họa dữ liệu

1.2. Xây dựng cấu trúc cây thư mục

Để chạy được chương trình trước tiên chúng ta sẽ tiến hành xây dựng cây thư mục cho ứng dụng:



Thư mục static sẽ chứa file **style.css** là file làm đẹp cho giao diện chính

Thư mục templates chứa file **index.html** giao diện chính cho chương trình

Bên ngoài chúng ta có:

- Intents.json: nơi chứa tệp dữ liệu có các mẫu và phản hồi được định trước

- Train_chatbot.py: xây dựng mô hình và đào tạo chatbot
- Words.pkl: tệp nhỏ trong đó sẽ lưu trữ các từ đối tượng Python chứa danh sách các từ vựng
- Classes.pkl: tệp chọn lớn chứa danh sách các danh mục
- Chatbot_model.h5: đây là mô hình đào tạo có chứa thông tin về mô hình và trọng số của các neurons
- Chatgui.py: là file thực thi chính trong cây thực mục này

III. QUY TRÌNH CHUẨN HÓA VÀ XỬ LÝ DỮ LIỆU

1.1. Năm bước chính để thực hiện

Bước 1: Import và load tệp dữ liệu

Bước 2: Tiền xử lý dữ liệu

Bước 3: tạo tệp dữ liệu đào tạo và kiểm tra

Bước 4: xây dựng mô hình

Bước 5: kiểm tra kết quả

1.2. Quy trình chuẩn hóa và xử lý

1.2.1. Import và load tệp dữ liệu

Đầu tiên, đặt tên tệp là `train_chatbot.py`. Import các thư viện cần thiết cho chatbot và khởi tạo các biến sẽ sử dụng trong project.

Tệp dữ liệu có dạng Json nên chúng ta sẽ import gói Json để phân tích.

```
train_chatbot.py > ...
1  import nltk
2  from nltk.stem import WordNetLemmatizer
3  lemmatizer = WordNetLemmatizer()
4  import json
5  import pickle
6
7  import numpy as np
8  from keras.models import Sequential
9  from keras.layers import Dense, Activation, Dropout
10 from keras.optimizers import SGD
11 import random
12
13 words=[]
14 classes = []
15 documents = []
16 ignore_words = ['?', '!']
17 data_file = open('intents.json',encoding="utf8").read()
18 intents = json.loads(data_file)
19
```

Cấu trúc file .JSON

```
intents.json > ...
1  [{"intents": [
2    {
3      "tag": "lời chào",
4      "patterns": ["xin chào", "hi", "."],
5      "responses": ["chào mừng bạn đến với cổng thông tin tuyển sinh của trường Đại học công nghiệp tphcm. Bạn có thắc mắc gì về thông tin tuyển sinh có thể",
6      "context": [""]
7    },
8    {
9      "tag": "tuyển sinh",
10     "patterns": ["tuyển sinh", "thông tin tuyển sinh", "phương thức tuyển sinh", "Cách tuyển sinh"],
11     "responses": ["Trường có 4 phương thức tuyển sinh. cụ thể như sau:\n1) Xét tuyển thẳng và ưu tiên xét tuyển thẳng đối với học sinh giỏi cấp tỉnh/thành phố",
12     "context": [""]
13   },
14   {
15     "tag": "đối tượng",
16     "patterns": ["đối tượng tuyển sinh", "đối tượng"],
17     "responses": ["sinh viên đã tốt nghiệp trung học phổ thông"],
18     "context": [""]
19   },
20   {
21     "tag": "thủ tục xét tuyển phương thức 1",
22     "patterns": ["thủ tục xét tuyển phương thức 1", "thủ tục xét tuyển thẳng"],
23     "responses": ["Hồ sơ gồm có:\n-Phiếu đăng ký xét tuyển (theo mẫu trên website của Trường);\n-01 Bản photocopy Học bạ THPT (có công chứng);\n-Giấy tờ",
24     "context": [""]
25   },
26   {
27     "tag": "thủ tục xét tuyển phương thức 2",
28     "patterns": ["thủ tục xét tuyển phương thức 2", "thủ tục xét tuyển kết quả học tập THPT lớp 12"],
29     "responses": ["Hồ sơ:\n-Phiếu đăng ký xét tuyển;\n-01 Bản photocopy Học bạ THPT (có công chứng);\n-01 Bản photocopy Bảng tốt nghiệp THPT (có công chứng)",
30     "context": [""]
31   },
32   {
33     "tag": "thủ tục xét tuyển phương thức 3",
34     "patterns": ["thủ tục xét tuyển phương thức 3", "thủ tục xét tuyển sử dụng kết quả kỳ thi tốt nghiệp"],
35     "responses": ["Hồ sơ, lệ phí đăng ký xét tuyển theo quy định của Bộ Giáo dục và Đào tạo, với mã tuyển sinh của Trường là IUH."],
36     "context": [""]
37   }
38 ]}]
```

1.2.2. Tiền xử lý dữ liệu

Khi làm việc với dữ liệu văn bản, chúng ta cần thực hiện tiền xử lý trên dữ liệu trước khi chúng ta tạo ra một mô hình ML hoặc DL. Tokenizing là điều cơ bản nhất và đầu tiên bạn có thể làm trên dữ liệu văn bản. Tokenizing là quá trình chia toàn bộ văn bản thành các phần nhỏ như các từ.

```
21 ∨ for intent in intents['intents']:
22 ∨     for pattern in intent['patterns']:
23
24         #tokenize each word
25         w = nltk.word_tokenize(pattern)
26         words.extend(w)
27         #add documents in the corpus
28         documents.append((w, intent['tag']))
29
30         # add to our classes list
31 ∨     if intent['tag'] not in classes:
32         classes.append(intent['tag'])
33
```

Bây giờ chúng ta sẽ bổ sung từng từ và xóa các từ trùng lặp khỏi danh sách.

```
34 # lemmatize and lower each word and remove duplicates
35 words = [lemmatizer.lemmatize(w.lower()) for w in words if w not in ignore_words]
36 words = sorted(list(set(words)))
37 # sort classes
38 classes = sorted(list(set(classes)))
39 # documents = combination between patterns and intents
40 print (len(documents), "documents")
41 # classes = intents
42 print (len(classes), "classes", classes)
43 # words = all words, vocabulary
44 print (len(words), "unique lemmatized words", words)
45
46
47 pickle.dump(words,open('words.pkl','wb'))
48 pickle.dump(classes,open('classes.pkl','wb'))
49
```

1.2.3. Tạo tập dữ liệu đào tạo về kiểm tra

Tiếp theo, chúng ta sẽ tạo tập training bao gồm các input và output, input là các mẫu còn output là các câu trả lời tương ứng. Tuy nhiên, máy tính không thể hiểu văn bản nên ta sẽ thực hiện chuyển văn bản dạng chữ về dạng số.

```

50 # create our training data
51 training = []
52 # create an empty array for our output
53 output_empty = [0] * len(classes)
54 # training set, bag of words for each sentence
55 for doc in documents:
56     # initialize our bag of words
57     bag = []
58     # list of tokenized words for the pattern
59     pattern_words = doc[0]
60     # lemmatize each word - create base word, in attempt to represent related words
61     pattern_words = [lemmatizer.lemmatize(word.lower()) for word in pattern_words]
62     # create our bag of words array with 1, if word match found in current pattern
63     for w in words:
64         bag.append(1 if w in pattern_words else bag.append(0))
65
66     # output is a '0' for each tag and '1' for current tag (for each pattern)
67     output_row = list(output_empty)
68     output_row[classes.index(doc[1])] = 1
69
70     training.append([bag, output_row])
71 # shuffle our features and turn into np.array
72 random.shuffle(training)
73 training = np.array(training)
74 # create train and test lists. X - patterns, Y - intents
75 train_x = list(training[:,0])
76 train_y = list(training[:,1])
77 print("Training data created")
78

```

1.2.4. xây dựng mô hình

Sau khi có được bộ training set, chúng ta sẽ thực hiện xây dựng 1 mạng neural với 3 layers bằng cách sử dụng Keras sequential API. Sau khi training mô hình với 200 epochs, mô hình đã đạt độ chính xác lên đến 91%. Tiếp theo chúng ta sẽ lưu mô hình dưới dạng ‘chatbot_model.h5’.

```

79
80 # Create model - 3 layers. First layer 128 neurons, second layer 64 neurons and 3rd output layer contains number of neurons
81 # equal to number of intents to predict output intent with softmax
82 model = Sequential()
83 model.add(Dense(128, input_shape=(len(train_x[0]),), activation='relu'))
84 model.add(Dropout(0.5))
85 model.add(Dense(64, activation='relu'))
86 model.add(Dropout(0.5))
87 model.add(Dense(len(train_y[0]), activation='softmax'))
88
89 # Compile model. Stochastic gradient descent with Nesterov accelerated gradient gives good results for this model
90 sgd = SGD(lr=0.01, decay=1e-6, momentum=0.9, nesterov=True)
91 model.compile(loss='categorical_crossentropy', optimizer=sgd, metrics=['accuracy'])
92
93 #fitting and saving the model
94 hist = model.fit(np.array(train_x), np.array(train_y), epochs=200, batch_size=5, verbose=1)
95 model.save('chatbot_model.h5', hist)
96
97 print("model created")
98

```

Độ chính xác sau khi chạy mô hình qua 200 epochs

```

...
25/25 [=====] - 0s 751us/step - loss: 0.1997 -
accuracy: 0.9344
Epoch 200/200
25/25 [=====] - 0s 875us/step - loss: 0.2267 -
accuracy: 0.9180

```

1.2.5. Dự đoán phản hồi (trên giao diện người dùng)

Bây giờ để dự đoán các câu và nhận được phản hồi từ người dùng, chúng ta sẽ tạo một tệp mới là 'chatgui.py'. Chúng ta sẽ load mô hình được đào tạo và sau đó sử dụng giao diện người dùng đồ họa (GUI) để dự đoán phản hồi từ bot. Mô hình sẽ chỉ cho chúng ta biết lớp mà nó thuộc về, vì vậy chúng ta sẽ triển khai một số hàm để xác định lớp và sau đó lấy cho chúng ta một phản hồi ngẫu nhiên từ danh sách các phản hồi. Một lần nữa, chúng ta import các gói cần thiết và tải các tệp pickle 'words.pkl' và 'class.pkl' mà chúng ta đã tạo khi đào tạo mô hình của mình:

```
chatgui.py > bow
1 import nltk
2 from nltk.stem import WordNetLemmatizer
3 lemmatizer = WordNetLemmatizer()
4 import pickle
5 import numpy as np
6
7 from keras.models import load_model
8 model = load_model('chatbot_model.h5')
9 import json
10 import random
11 # Thêm Mới
12 from flask import Flask, render_template, request
13 # Thêm Mới
14 intents = json.loads(open('intents.json', encoding="utf8").read())
15 words = pickle.load(open('words.pkl', 'rb'))
16 classes = pickle.load(open('classes.pkl', 'rb'))
17
```

Để dự đoán lớp, chúng ta cần cung cấp thông tin đầu vào giống như cách chúng ta đã làm khi training. Vì vậy, chúng ta sẽ tạo một số hàm để thực hiện tiền xử lý văn bản và sau đó dự đoán lớp.

```
28 def bow(sentence, words, show_details=True):
29     # tokenize the pattern
30     sentence_words = clean_up_sentence(sentence)
31     # bag of words - matrix of N words, vocabulary matrix
32     bag = [0]*len(words)
33     for s in sentence_words:
34         for i,w in enumerate(words):
35             if w == s:
36                 # assign 1 if current word is in the vocabulary position
37                 bag[i] = 1
38                 if show_details:
39                     print("found in bag: %s" % w)
40     return(np.array(bag))
41
42 def predict_class(sentence, model):
43     # filter out predictions below a threshold
44     p = bow(sentence, words, show_details=False)
45     res = model.predict(np.array([p]))[0]
46     ERROR_THRESHOLD = 0.25
47     results = [(i,r) for i,r in enumerate(res) if r>ERROR_THRESHOLD]
48     # sort by strength of probability
49     results.sort(key=lambda x: x[1], reverse=True)
50     return_list = []
51     for r in results:
52         return_list.append({"intent": classes[r[0]], "probability": str(r[1])})
53     return return_list
54
```

Sau khi dự đoán được lớp, ta sẽ nhận được phản hồi ngẫu nhiên từ danh sách intents.

```
55 ∨ def getResponse(ints, intents_json):
56     tag = ints[0]['intent']
57     list_of_intents = intents_json['intents']
58     for i in list_of_intents:
59         if(i['tag']== tag):
60             result = random.choice(i['responses'])
61             break
62     return result
63
64 ∨ def chatbot_response(msg):
65     ints = predict_class(msg, model)
66     res = getResponse(ints, intents)
67     return res
68
```

CHƯƠNG 4 XÂY DỰNG MÔ HÌNH VÀ ĐÁNH GIÁ

I. XÂY DỰNG MÔ HÌNH

1.1. xây dựng mô hình

```
80 # Create model - 3 layers. First layer 128 neurons, second layer 64 neurons and 3rd output layer contains number of neurons
81 # equal to number of intents to predict output intent with softmax
82 model = Sequential()
83 model.add(Dense(128, input_shape=(len(train_x[0]),), activation='relu'))
84 model.add(Dropout(0.5))
85 model.add(Dense(64, activation='relu'))
86 model.add(Dropout(0.5))
87 model.add(Dense(len(train_y[0]), activation='softmax'))
88
89 # Compile model. Stochastic gradient descent with Nesterov accelerated gradient gives good results for this model
90 sgd = SGD(lr=0.01, decay=1e-6, momentum=0.9, nesterov=True)
91 model.compile(loss='categorical_crossentropy', optimizer=sgd, metrics=['accuracy'])
92
93 #fitting and saving the model
94 hist = model.fit(np.array(train_x), np.array(train_y), epochs=200, batch_size=5, verbose=1)
95 model.save('chatbot_model.h5', hist)
96
```

Bước 1: Khởi tạo một tập đầu vào chiều dài vào bằng với chiều dài tập Input

Bước 2: Xây dựng một Hidden Layer với chiều sâu bằng 128 neurons và hàm kích hoạt là 'relu'

Bước 3: Thực hiện Dropout với tỉ lệ 0.5 tập dữ liệu để chống over-fitting (một hiện tượng không mong muốn thường gặp)

Bước 3: Xây dựng một Hidden Layer thứ 2 với chiều sâu là 64 neurons và hàm kích hoạt là 'relu'

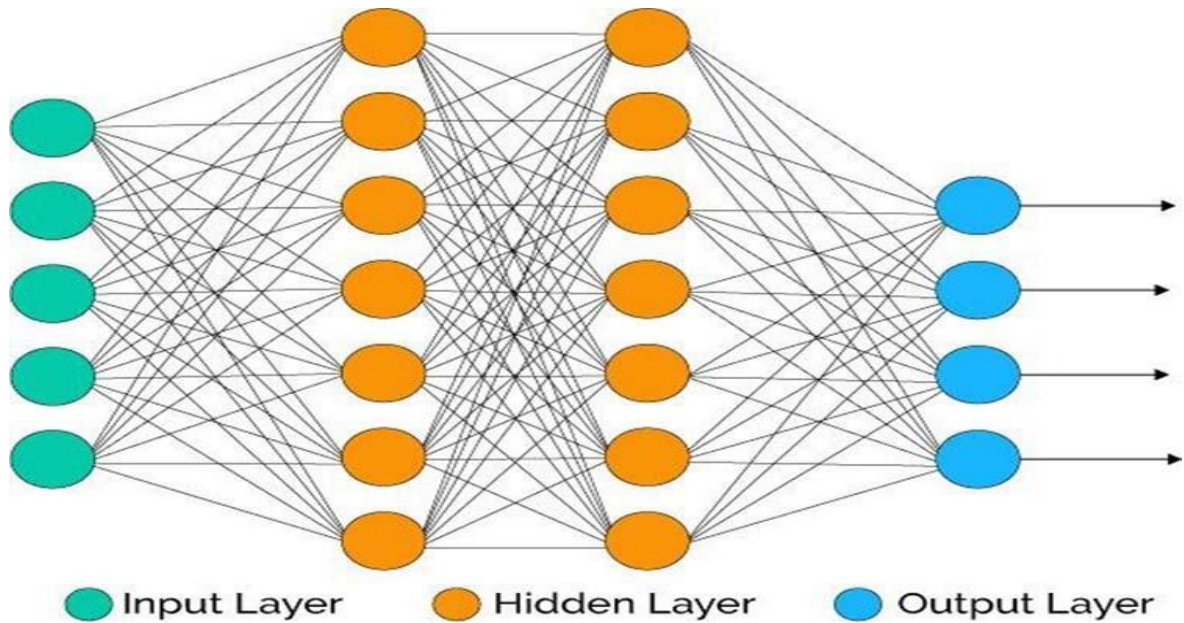
Bước 4: Thực hiện Dropout như bước 3

Bước 5: Xây dựng một Hidden Layer thứ 3 với chiều sâu bằng với độ dài tập train và hàm kích hoạt là 'softmax'

1.2. Tối ưu cho mô hình

Sử dụng hàm SGD với các thông số như: lr = 0.01, decay = 1e-6, momentum = 0.9, momentum = 0.9, nesterov = True.

Sơ đồ mạng Neural



ảnh minh họa

II. ĐÁNH GIÁ MÔ HÌNH

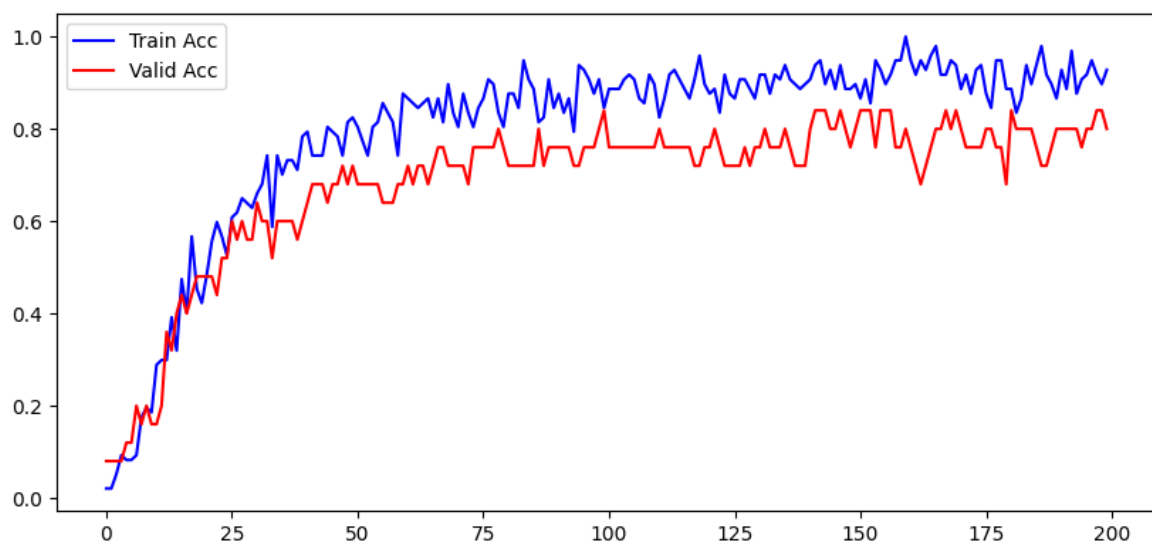
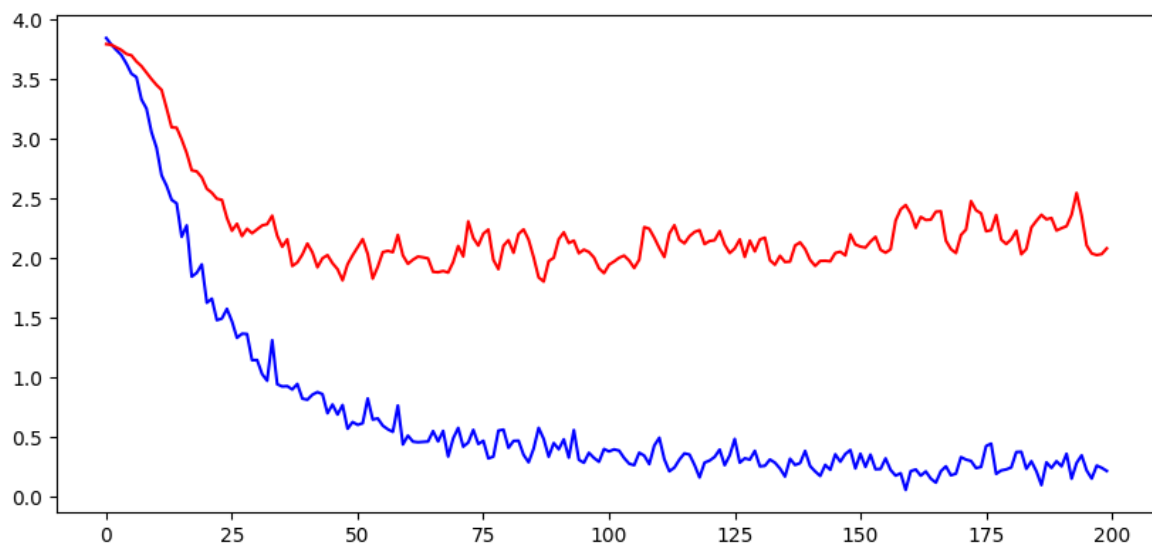
1.1. Loss Function

Đơn giản nhất, ta sử dụng Tổng độ lệch bình phương làm hàm mất mát:

$$\text{Sum - of - Squares Error} = \sum_{i=1}^n (y - \hat{y})^2$$

Tổng độ lệch bình phương là tổng số sai khác giữa giá trị dự đoán và giá trị thực tế. Giá trị này được bình phương giúp chúng ta có thể đo được sự sai khác tuyệt đối.

Mục tiêu của chúng ta là tìm ra được hàm Loss Function có giá trị nhỏ nhất.



CHƯƠNG 5 THỰC NGHIỆM VÀ ĐÁNH GIÁ

I. THỰC NGHIỆM HỆ THỐNG

Hệ thống chạy ổn định, không lỗi. Dữ liệu cho model cần dữ liệu nhiều hơn để có thể cho ra kết quả tốt nhất model khi visualize qua loss function cho kết quả đang tốt dần lên.

II. ĐÁNH GIÁ QUÁ TRÌNH HOẠT ĐỘNG CHO HỆ THỐNG

Sau khi trải qua quá trình triển khai và thực nghiệm hệ thống, Ứng dụng chatbots AI đã đáp ứng được những nhu cầu của nhà trường về vấn đề tuyển sinh. Hệ thống đưa ra những câu trả lời chính xác với những câu hỏi, tốc độ xử lý nhanh

III. KIỂM THỬ VÀ CÁC VẤN ĐỀ GẶP PHẢI TRONG QUÁ TRÌNH THỰC HIỆN

Tuy nhiên, với những yêu cầu phức tạp đòi hỏi những đáp ứng chuyên sâu hơn và những phản hồi mang tính cụ thể, chẳng hạn như các vấn đề liên quan đến độ chính xác cao, các con số, các dữ liệu biến động,...lúc này chatbots sẽ cần phải được kết nối với cơ sở dữ liệu của đối tượng tương ứng để đáp ứng được yêu cầu của người dùng. quá trình này được gọi là việc xử lý các ý định nghiệp vụ. Ngoài ra hệ thống cần được bổ sung thêm dữ liệu để có thể hoàn thiện hơn.

- Kết luận

Sau khi trải qua quá trình tìm hiểu và thực hiện đề tài xây dựng Chatbots AI hỗ trợ tuyển sinh của trường Đại học Công Nghiệp TPHCM nhóm đã thực hiện được những yêu cầu cơ bản ban đầu đưa ra và nắm được những lý thuyết như: khái niệm về Chatbots AI, kiểu máy học có giám sát (Supervised machine learning) , thu thập dữ liệu, xử lý dữ liệu, số hóa dữ liệu, và xây dựng mô hình để huấn luyện. Ngoài ra nhóm cũng đã truyền khai hệ thống lên môi trường web và có thể thực hiện được. Bên cạnh đó hệ thống cũng có nhiều hạn chế, mô hình huấn luyện còn thô sơ chưa đạt được độ chính xác như mong muốn vì nhóm nhóm có thể nghiên cứu và phát triển trong tương lai.

- Phương hướng phát triển:

Thấy được những hạn chế của hệ thống, nhóm sẽ phát triển hệ thống bằng cách bổ sung thêm dữ liệu, cải tiến mô hình để có thể cho ra dữ liệu có độ chính xác cao và linh hoạt hơn.

